

Представление данных

Рекап

- Как в компьютере представлены числа?
- Что если число занимает больше одного байта?
- Что такое адрес?
- Что такое тип данных?
- Какие типы данных вы знаете, чем они отличаются?
- Чем тип строки отличается от типа целого числа

Типы данных

Переменная – именованная область памяти.

Тип данных – это информация о размере переменной и доступных операциях с ней.

Типы данных

Примеры из C:

- int:
 - целые
 - 4 байта
 - операции: +, -, *, /, ...
- char:
 - целые
 - 1 байт
 - операции: +, -, *, /, ...
- float:
 - floating point ieee754
 - 4 байта.
 - операции: +, -, *, /, ...
- void*:
 - указатель,
 - 8 байт.
 - операции: cast к другому типу

Типы данных

Сосредоточимся на размерах

- Прошлый слайд вам нагло врет
- Согласно стандарту, размер `int` – любой (от 2 байт)
- Это нужно, чтоб на разных компьютерах код работал максимально эффективно
- Аналогично для множества других типов

Но как узнать размер типа конкретно у вас?

Типы данных

Но как узнать размер типа конкретно у вас?

В языке C для этого существует оператор `sizeof(type)`

Это оператор резолуции времени компиляции, т.е. он вычисляется во время компиляции, еще до запуска программы.

Компилятор, очевидно, знает конкретные размеры типов на вашей машине и может их подставить во время компиляции.

Массивы

Массив – это структура *однотипных* данных, занимающих *непрерывную* область памяти

Главная особенность массива в том, что данные лежат непрерывно

Зная адрес начала и размер элемента, мы можем получить доступ к *i*-му элементу:

```
a[i] = *(base_address + i*sizeof(a[0]))
```

Отступление. Арифметика указателей

При сложении указателей, домножение на размер типа происходит автоматически.

Размер типа берется из типа указателя.

Читать K&R!!11!!!1!!

Отладка

Мы уже научились смотреть на размеры, адреса и значения данных в коде.

До сих пор, мы писали отдельные программы с `printf`, чтоб это сделать.

Но что если у нас есть огромная программа, которая не работает как надо – то есть в ней содержится баг. Мы хотим распечатать значения переменных, чтоб понять, что приводит к ошибке. Как это сделать проще всего?

Отладка

Но что если у нас есть огромная программа, которая не работает как надо – то есть в ней содержится баг. Мы хотим распечатать значения переменных, чтоб понять, что приводит к ошибке. Как это сделать проще всего?

- Не писать программы с багами.

Отладка

Но что если у нас есть огромная программа, которая не работает как надо – то есть в ней содержится баг. Мы хотим распечатать значения переменных, чтоб понять, что приводит к ошибке. Как это сделать проще всего?

- Не писать программы с багами.
- Расставить `printf` по всему коду

Отладка

Но что если у нас есть огромная программа, которая не работает как надо – то есть в ней содержится баг. Мы хотим распечатать значения переменных, чтоб понять, что приводит к ошибке. Как это сделать проще всего?

- Не писать программы с багами.
- Расставить `printf` по всему коду
- Использовать специальные программы-отладчики (дебаггеры)

Дебаггеры

Дебаггер – это программа необходимая для анализа работы вашей программы.

Некоторые возможности дебаггеров:

- построчное исполнение кода (исполняем одну строчку и “ставим на паузу”)
- печать значений переменных
- инспекция памяти (печать данных по какому-то адресу)
- расстановка брейкпойнтов (когда выполнение доходит до брейкпойнта, исполнение “ставится на паузу”)

И много других

GDB / LLDB

Дебаггеров есть очень много и вы даже можете написать свой. Есть дебаггеры, специфичные для конкретных языков, есть общие. Пример:

- pydb – дебаггер python
- gdb – gnu debugger – общего назначения, работает с C, C++, Rust и т. д.
- lldb – llvm debugger – общего назначения, работает с C, C++, Rust и т. д.

GDB

Будем работать с gdb как с более популярным. Команды lldb часто похожи.

Разберем несколько команд:

- компиляция программы для отладки
- запуск программы
- установка брейкпойнта
- исследование переменных
- исследование памяти

<https://habr.com/ru/articles/1016430/>

<https://habr.com/ru/articles/535960/>

GDB. Компиляция и запуск дебаггера

Компилятор делает оптимизации и выбрасывает часть информации о коде для уменьшения размера файла. Это ухудшает отладку. Мы должны указать специальные флаги, чтоб компилятор этого не делал и, наоборот, добавил нам информации о коде для отладки.

```
gcc -g -O0 test.c -o prog
```

```
gdb ./prog
```

```
(gdb) run
```

GDB. Установка брейкпойнтов

Брейкпойнт – точка, где дебаггер поставит исполнение “на паузу”, чтоб вы могли выполнить другие команды и продолжить исполнение

- `break main (b main)`: на функции
- `break prog.c:42`: на строке
- `info breakpoints, delete 1`: список / удалить
- `continue (c)`: продолжить до следующей точки

GDB. Пошаговое выполнение

Выполняем по одной строчке кода за раз.

- `next (n)`: следующая строка, не заходя в функции
- `step (s)`: следующая строка, заходя в функции
- `finish`: выполнить до выхода из текущей функции

GDB. Исследование памяти и переменных

Переменные

- `print x (p x), p arr[3], p *ptr, p s.field`
- `p/x x`: в шестнадцатеричном виде
- `info locals, info args`: все локальные переменные / аргументы
- `display x`: печатать x после каждого шага

Память и регистры

- формат: `x/<кол-во><формат><размер>`; форматы `x d c s i`, размеры `b h w g`
- `x/8xw &arr`: 8 слов по 4 байта в hex
- `x/s str`: строка
- `x/10xb ptr`: 10 байт

Работаем в терминале

Терминал – программа для работы с компьютером. Раньше это было физическое устройство для подключения к мейнфрейму.

- Раньше ничего кроме терминала не было
- Терминалы UNIX-подобных систем (в т.ч. Linux) невероятно мощные. Ими можно полностью управлять компьютером
- Терминал всегда находится в какой-то конкретной папке.

- `pwd` (print working directory) – Текущая папка
- `cd <путь>` (change directory) – Сменить текущую папку
- `ls` (list) – Вывести содержимое текущей папки
- `whoami` – Кто я
- `touch grass` – потрогать траву

Литература

- КеРнИгаН и РиТччи!!1!11!!!!1
- Статьи по GDB:
 - <https://habr.com/ru/articles/1016430/>
 - <https://habr.com/ru/articles/535960/>
- Статьи про shell (кюд привет переведи статью)
 - <https://missing.csail.mit.edu/2020/course-shell/>
 - https://linuxcommand.org/lc3_learning_the_shell.php